Reliable Hybrid SLC/MLC ReRAM Crossbars for Transformers

Component Placement, Fault-Aware Mapping and Noise-Aware Training

Shivansh Mittal Sarvesh Joshi Thumati Naveen Chandra Chowdary Aryan Chakravorty
Indian Institute of Technology Guwahati
Course Project, CS5290 Memory-Centric Computer Architecture

***Abstract*—Hybrid ReRAM crossbars keep the most important weights in reliable single-level cells (SLC) and the rest in dense multi-level cells (MLC). HyFlexPIM applies this to transformers by factorising each weight matrix with SVD and protecting the components with the largest loss gradients. We evaluate this scheme on BERT-Base and DistilBERT (SST-2) with a bit-accurate crossbar simulator that models errors cell by cell: programming and read noise, level-dependent noise, stuck cells, a fixed ADC with bit-serial inputs and IR drop. A hybrid layout is necessary, since all-MLC storage falls to 52–60% accuracy at 7.5% programming noise, but the choice of components matters as much as their number. Ranking components by singular value reaches the same accuracy as a Fisher-information estimate of each component's expected damage, while a gradient score loses 2–12 points at high noise; we explain this through the per-column quantisation of the crossbar. With magnitude ranking, 5–10% of components in SLC match an all-SLC chip at 5% noise. Stuck cells are the one effect placement cannot handle. Because the components of an SVD layer can be stored in any order, we map them to physical slots around a tested chip's stuck cells by solving an assignment problem, at no hardware cost; this raises BERT-Base at 0.5% stuck cells from 76.6% to 85.6%. Fine-tuning on simulated chips with stuck cells adds to this mapping. A tier-aware variant, which trains each component under the errors of the tier (SLC or MLC) it will be stored in, lifts BERT-Base at 1% stuck cells from 65.1% to 82.9%, so that the hybrid chip matches or exceeds an all-SLC chip with mapping up to 1% scattered stuck cells, using 34% fewer cells. The placement and mapping results also hold on MNLI.**

***Index Terms*—ReRAM, processing-in-memory, multi-level cell, stuck-at faults, SVD, transformer, fault-aware mapping, noise-aware training**

I. Introduction

RESISTIVE RAM (ReRAM) crossbars compute a matrix–vector product where the weights are stored. Each weight is held as the conductance of one or more resistive cells; applying the input as row voltages makes each column current the dot product of the input with that column's conductances [1], [2]. Weights never move between memory and compute, which removes most of the data movement that dominates the energy of neural-network inference on conventional hardware. The cost is accuracy: a cell cannot be programmed to an exact conductance, its reading fluctuates, and some cells are permanently stuck at their highest or lowest conductance.

A cell can store one bit (single-level cell, SLC) or several bits (multi-level cell, MLC). MLC is denser, but its conductance levels are closer together, so the same conductance error corrupts more of the stored value. HyFlexPIM [3] combines the two for transformers: it factorises each weight matrix with singular value decomposition (SVD), keeps the components with the largest loss gradients in SLC and stores the rest in MLC (Section II). Its evaluation applies relative noise to whole weights at a single noise level.

We revisit this design with errors modelled cell by cell, as they arise in hardware, and ask which components should be protected, how many, and which hardware effects still break the scheme. We study BERT-Base [4] and DistilBERT [5] on the SST-2 sentiment task [6], [7], and repeat the placement and mapping study on MNLI.

Our contributions. We keep HyFlexPIM's SVD factorisation and hybrid layout unchanged and add the following.

- **Cell-level evaluation** (Section III). HyFlexPIM perturbs whole weights at one noise level. We build a verified crossbar model that adds noise to every cell of a differential pair and its bit slices, sweep programming noise from 0 to 10%, and add read noise, level-dependent MLC noise, three patterns of stuck cells, a fixed ADC with bit-serial inputs and IR drop. Every configuration is compared on the same simulated chips.
- **Which components to protect, and why** (Sections IV and X). HyFlexPIM ranks components by their loss gradient. We show that ranking by singular value is as good as a Fisher-information estimate of each component's expected damage and better than the gradient at high noise, and explain this through the per-column quantisation of the crossbar.
- **Stuck cells as the limiting effect.** HyFlexPIM does not model stuck cells. We find that every other effect leaves the magnitude hybrid within about a point of all-SLC, while stuck cells break every placement.
- **Fault-aware mapping for SVD layers** (Section V). The components of an SVD layer can be stored in any order. We use this freedom to place components around a tested chip's stuck cells, by a Fisher-weighted assignment of components to slots, with no extra hardware or retraining.
- **Tier-aware training** (Section VI). In HyFlexPIM, hardware noise is applied only at inference, not during fine-tuning. We train on simulated chips, with each component

under the errors of the tier (SLC or MLC) it will be stored in. Together with mapping, this lets the hybrid chip match an all-SLC chip with mapping up to 1% scattered stuck cells, using 34% fewer cells.

Section II reviews background and related work, Sections III to VI describe the model and methods, Sections VII to IX the experiments, and Sections X and XI discuss and conclude.

II. Background and Related Work

A. Matrix–Vector Products in a ReRAM Crossbar

A crossbar places a resistive cell at every crossing of a row (word line) and a column (bit line). If cell (i, j) has conductance G_{ij} and row i is driven at voltage V_i, the current collected at column j is

$$I_j = \sum_i V_i \, G_{ij}, \tag{1}$$

by Ohm's law in each cell and Kirchhoff's current law on the column. One read therefore computes a full matrix–vector product in the analog domain. Accelerators such as ISAAC [1] and PRIME [2] build on this primitive. Three conventions from these designs are used throughout this report:

- Conductances are non-negative, so a signed weight is stored as the difference of two cells, a *differential pair*.
- A cell holds only a few bits, so a multi-bit weight is split into *bit slices* on separate cells; the column results of the slices are shifted and added digitally.
- Arrays have a fixed size (we use 128×128 tiles), and an analog-to-digital converter (ADC) digitises each column current.

B. SLC, MLC and Non-Idealities

A cell programmed to $L = 2^b$ levels between $G_{\min}$ and $G_{\max}$ stores b bits; $b = 1$ is a single-level cell (SLC) and $b \geq 2$ a multi-level cell (MLC). The level spacing is $\Delta = (G_{\max} - G_{\min})/(L - 1)$, so a two-bit cell has levels three times closer than an SLC. A conductance error of fixed size is therefore three times larger relative to an MLC level, while MLC needs fewer cells per weight. Real arrays deviate from the programmed state in several ways:

- *Programming variation*: each cell lands near, not on, its target conductance; the error is fixed once the chip is programmed.
- *Read noise*: the current fluctuates at every read.
- *Stuck-at faults*: some cells are permanently stuck at $G_{\max}$ or $G_{\min}$, whatever is written; faults can be scattered, clustered, or affect a whole line.
- *IR drop*: wire resistance lowers the voltage reaching cells far from the drivers.
- *ADC resolution*: column currents are rounded to a limited number of levels.

C. HyFlexPIM

HyFlexPIM [3] is a mixed-signal processing-in-memory accelerator for transformers. Its analog part can switch each array between SLC and MLC operation, so that critical weights use SLC and the rest use denser, cheaper MLC. Placing raw weights this way gives little benefit, because importance is spread over many weights. HyFlexPIM therefore factorises each weight matrix with a truncated SVD, $W \approx U\Sigma V^\top$, and fine-tunes the factors, which concentrates the loss gradient in a small subset of components; it calls this *gradient redistribution*. Components are ranked by the gradient of the loss with respect to their singular value, and the top-ranked share is mapped to SLC. Accuracy is evaluated with relative Gaussian noise applied to whole weights, $\tilde{W} = W \odot (1+\eta)$, at a single level derived from a measured MLC bit error rate [8], together with a limited ADC resolution. Stuck cells, read noise and IR drop are not modelled. Our work keeps the SVD factorisation and the hybrid layout unchanged; Section I lists what it adds.

D. Fault-Tolerant Mapping and Training

Two families of methods make crossbar inference tolerant to device errors. The first adapts training to the hardware. Chen *et al.* [9] include device variations and defects in training. Xia *et al.* [10] detect stuck cells on line and adapt training to them. Rasch *et al.* [11] inject hardware noise during training at scale, in what is now called hardware-aware training; gradients pass through the non-differentiable hardware model with a straight-through estimator [12]. The second family leaves the weights unchanged and chooses *where* they are stored, so that important values avoid faulty cells. Such mapping problems reduce to assignment problems, which the Hungarian method [13] solves exactly.

Our fault-aware mapping belongs to the second family, with two differences. First, the freedom to reorder comes from the SVD itself: the components of a factorised layer can be stored in any order without changing its output, so no permutation hardware or change to neighbouring layers is needed. Second, the cost of a placement is weighted by the Fisher information of each stored value, a second-order estimate of loss sensitivity in the spirit of Optimal Brain Damage [14] and of Fisher-based importance in continual learning [15].

III. Crossbar Model

We wrote a PyTorch simulator that replaces each linear layer of a model with crossbars and models the stored value of every cell. This section describes how a weight matrix is stored, the error sources, and how the simulator was verified. Table I lists the parameters.

A. Storing a Weight Matrix

A matrix $W \in \mathbb{R}^{m \times n}$, with n inputs and m outputs, is stored with its inputs on the rows (word lines) and its outputs on the columns (bit lines) of the crossbar. It is quantised to signed 8-bit integers with one scale per output column,

$$q_{ij} = \mathrm{clip}\big(\,\mathrm{round}(w_{ij}/s_i),\, -127,\, 127\big), \tag{2}$$

TABLE I
CROSSBAR MODEL PARAMETERS.

Parameter	Value
Weight precision	8-bit signed, one scale per output column
Input precision	8-bit signed
Cells per weight (per sign)	7 SLC ($b = 1$) or 4 MLC ($b = 2$)
Conductance range	$G_{\max} = 1$ (normalised), $G_{\max}/G_{\min} = 100$
Tile size	128×128
Programming noise σ	0–10% of $G_{\max}$, fixed per chip
Read noise σ_r	0.5–5% of $G_{\max}$, fresh at every read
Stuck cells p	0.1–5% of cells, half at $G_{\max}$, half at $G_{\min}$
ADC	ideal, or 3–8 bits with a fixed calibrated range
Wire resistance r	R_{wire} per segment / R_{on}, 10^{-4}–3×10^{-3}

with column scale $s_i = \max_j |w_{ij}|/127$ for output i. The sign is handled by a differential pair, $q = q^+ - q^-$ with $q^\pm = \max(\pm q, 0)$, and each 7-bit magnitude is split into $S = \lceil 7/b \rceil$ base-L digits,

$$q^\pm = \sum_{t=0}^{S-1} L^t d_t^\pm, \qquad d_t^\pm \in \{0, \ldots, L-1\}, \tag{3}$$

each stored in its own cell at conductance $G = G_{\min} + d\,\Delta$. An SLC weight thus uses 2×7 cells and an MLC weight 2×4. The stored weight is read back as

$$\hat{w}_{ij} = s_i \sum_{t=0}^{S-1} L^t \, \frac{G^+_{ij,t} - G^-_{ij,t}}{\Delta}, \tag{4}$$

where the $G_{\min}$ offset cancels in the difference. Without errors, $\hat{w}_{ij} = s_i q_{ij}$ exactly. Inputs are quantised to 8 bits in the same way with one scale per tensor, and the layer output is the product of the quantised input with $\hat{W}$.

B. Error Sources

Each error source is a separate, seeded option, so that its effect can be measured on its own.

- **Programming noise.** Every cell receives a fixed Gaussian offset, $\tilde{G} = \max\big(0,\ G + \sigma G_{\max}\varepsilon\big)$ with $\varepsilon \sim \mathcal{N}(0,1)$, drawn once per chip. Because Δ is three times smaller for MLC, the same σ causes a three times larger digit error in MLC.
- **Level-dependent noise.** The noise of an MLC cell can depend on its level: σ is multiplied by a factor m_d per level d. We use $m = (1, 1.5, 1.5, 1)$ and $m = (1, 2, 2, 1)$, making the two middle levels up to twice as noisy as the end levels.
- **Read noise.** An independent Gaussian offset of standard deviation $\sigma_r G_{\max}$ is added to every cell at every read. Since it is independent of the stored value, its effect on an output is Gaussian with a closed-form variance, which the simulator adds directly to the output.
- **Stuck cells.** A cell is stuck with probability p, at $G_{\max}$ or $G_{\min}$ with equal probability, and keeps that conductance whatever is written. In the *clustered* pattern, faults grow from random seeds into 3×3 blocks, with the seed rate set so that the overall rate is about p. In the *line* pattern, a whole bit line within one 128-row tile fails with probability p.
- **ADC and bit-serial inputs.** With a finite ADC, inputs are applied one bit at a time: each input is written in sign-magnitude form, and bit k of its magnitude is applied with the input's sign. For every input bit k, weight slice t and 128-row tile, the column sum is quantised by a b_{ADC}-bit ADC and the results are combined digitally,

$$y_j = \sum_k \sum_t 2^k L^t \, \mathrm{ADC}_t\Big(\sum_{i \in \text{tile}} x_i^{(k)} \big(d^+_{ij,t} - d^-_{ij,t}\big)\Big). \tag{5}$$

  The full-scale range of each slice's ADC is fixed once, from the largest column sum observed on 64 training sentences on an error-free chip, and is not adjusted at run time.
- **IR drop.** A first-order static model attenuates each cell by its distance from the row driver and the column sense amplifier. A cell crossed by n wire segments behaves as

$$G_{\text{eff}} = \frac{G}{1 + r\,G\,n}, \qquad n = (c+1) + (127 - \rho), \tag{6}$$

  where c and ρ are the cell's column and row positions within its tile, and r is the resistance of one wire segment relative to the on-resistance. This ignores the dependence of IR drop on the input pattern, which is enough to show whether a cell's position matters.

C. Chips and Paired Comparison

A *chip* is one draw of all random errors in a model: noise offsets, stuck cells and their values. All draws come from a single random generator seeded with the chip number, and every placement or mapping is evaluated on the same seeds. Two placements are therefore compared on chips with identical error draws, and their difference on each chip is a paired measurement. We report the mean accuracy over chips and, where differences are small, the mean paired difference with its standard error.

D. Verification

Every experiment starts with self-tests on random matrices, and stops if any check fails:

- With all errors disabled, the crossbar output equals the 8-bit integer product to a relative error below 10^{-4}, for SLC and MLC, both in one pass and when read slice by slice, tile by tile and bit by bit.
- The bit planes of an input rebuild it exactly.
- A calibrated 16-bit ADC is lossless to a relative error below 10^{-3}, while a 4-bit ADC is not.
- The measured spread of read noise matches its closed-form variance to within 15%.
- The three fault patterns produce the requested share of stuck cells.
- IR drop only reduces stored conductances, and level-dependent noise with all factors equal to one reproduces plain programming noise exactly.

End to end, a chip with no errors reaches within about one point of the digital model for every placement (Section VIII).

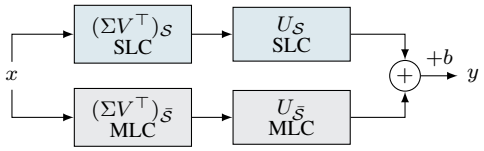


Fig. 1. A factorised layer on the chip. Components in $\mathcal{S}$ use SLC crossbars, the rest MLC crossbars; each component is one column of the first crossbar and one row of the second.

IV. SVD Tiering and Component Placement

A. Factorisation and Fine-Tuning

Following HyFlexPIM [3], every linear layer of the transformer encoders (72 matrices in BERT-Base, 36 in DistilBERT) is factorised with a truncated SVD,

$$W \approx U\Sigma V^\top = \sum_{r=1}^{k} \sigma_r\, u_r v_r^\top, \qquad k = \Big\lfloor \frac{mn}{m+n} \Big\rfloor, \tag{7}$$

for $W \in \mathbb{R}^{m\times n}$. This rank keeps the parameter count unchanged, e.g. $k = 384$ for a 768×768 attention matrix and $k = 614$ for a 768×3072 feed-forward matrix. Truncation loses accuracy, so the factorised model is fine-tuned on the full SST-2 training set (67,349 sentences) for 8 epochs with AdamW, learning rate 2×10^{-5} with linear decay, and batch size 32. U, Σ and V are trained directly, so the factors need not stay orthogonal.

B. Mapping a Factorised Layer onto Two Tiers

We call (σ_r, u_r, v_r) the r-th *component*. On the chip it is stored as one column of a first crossbar, holding $\sigma_r v_r$, and one row of a second crossbar, holding u_r: the first crossbar turns the input into one value per component, and the second spreads each value back over the outputs. A placement chooses a set $\mathcal{S}$ of components for SLC; the rest form the MLC tier. Each tier has its own pair of crossbars (Fig. 1), and the layer computes

$$y = U_{\mathcal{S}}\big(\Sigma V^\top\big)_{\mathcal{S}}\, x + U_{\bar{\mathcal{S}}}\big(\Sigma V^\top\big)_{\bar{\mathcal{S}}}\, x + b. \tag{8}$$

The intermediate vector between the two crossbars is requantised to 8 bits, as any crossbar input. A component holds $m+n$ weights, so placing a share f of the components in SLC costs $14f + 8(1-f)$ cells per weight against 14 for all-SLC; at $f = 20\%$ this is 9.2 cells, 34% fewer. All placements select the top $\lceil fk \rceil$ components of each layer by a score, so every layer has the same SLC share.

C. Placement Scores

Besides the two extremes, all-SLC and all-MLC, and a random choice as a control, we compare three scores. All data-dependent scores use the same 2,048 SST-2 training sentences.

Magnitude. The size of the singular value, $|\sigma_r|$ (σ_r is trained freely and need not stay positive). It needs no data.

Gradient (HyFlexPIM-style). The absolute loss gradient with respect to the singular value, summed over batches b of 32 sentences with dropout active,

$$g_r = \sum_b \Big| \frac{\partial L_b}{\partial \sigma_r} \Big|. \tag{9}$$

The absolute value is taken per batch because, on a trained model, the gradients of different sentences point in opposite directions and largely cancel in a single sum.

Fisher. The expected rise in loss if component r is stored in MLC. To second order, a small random error δ_e on stored entry e raises the expected loss by about $\frac{1}{2} F_e\, \mathbb{E}[\delta_e^2]$, where F_e is the Fisher information of that entry [14]. We estimate F_e by the empirical Fisher, the sum of squared per-sentence gradients. Under the cell model of Section III, a fixed cell noise becomes a weight error proportional to the 8-bit scale of the crossbar column holding the entry, so $\mathbb{E}[\delta_e^2] \propto s^2$. Summing over every entry the component stores gives

$$D_r = s_r^2 \sum_j F\big(\sigma_r v_{rj}\big) + \sum_i t_i^2\, F\big(u_{ir}\big), \tag{10}$$

where $s_r = \max_j |\sigma_r v_{rj}|/127$ is the scale of the component's column in the first crossbar and $t_i = \max_r |u_{ir}|/127$ that of output column i in the second. The constant factor of the noise variance is the same for every component and is omitted.

V. Fault-Aware Mapping

Placement decides which tier each component uses, but not which physical position in that tier. With stuck cells this matters: a stuck high-order cell shifts its weight by a large fixed amount, so it does far more harm under an important component than under an unimportant one. This section uses a property of the SVD layer to choose the position of every component around a chip's stuck cells.

A. Free Order of Components

For any permutation matrix P,

$$(UP)\big(P^\top \Sigma V^\top\big) = U\Sigma V^\top, \tag{11}$$

so the k components of a layer can be stored in any order without changing its output. We call the j-th position of a tier a *slot*: column j of the tier's first crossbar together with row j of its second crossbar. Moving a component to another slot moves its column $\sigma_r v_r$ and its row u_r together. The order is fixed when the chip is programmed, so a chosen mapping needs no permutation hardware and no change to the neighbouring layers.

B. Known Stuck Cells

Stuck cells can be located after fabrication by writing and reading back every cell. We assume this test has been done, so each chip's stuck-cell map is known before its weights are programmed; in simulation it is the chip's drawn fault map. Programming noise is random and not known in advance, so the mapping targets stuck cells only.

C. Cost of a Component in a Slot

Placing component r in slot s costs the expected rise in loss caused by the stuck cells of that slot. A cell of slice t stuck at conductance G^*, under an entry whose digit should be d, changes the entry by

$$\delta = \pm\, s\, L^t\, \frac{G^* - (G_{\min} + d\,\Delta)}{\Delta}, \tag{12}$$

with s the 8-bit scale of the crossbar column holding the entry and the sign set by the side of the differential pair. Weighting each error by the Fisher information of its entry, as in Eq. (10), gives

$$C_{rs} = \sum_{\text{stuck cells } c \text{ in slot } s} F_{e(r,c)}\, \delta_{r,c}^2, \tag{13}$$

where $e(r, c)$ is the entry of component r that would occupy cell c. The first-crossbar column moves with its component and keeps its own scale s_r, while the scales t_i of the second crossbar depend only on which components share the tier, so every term of C_{rs} is known before programming. The sum treats two stuck cells under the same entry as independent, which slightly misestimates their joint error; it is exact when each entry has at most one stuck cell.

D. Assignment

Within each tier, the mapping is the permutation π that minimises $\sum_r C_{r\pi(r)}$, a linear assignment problem. We solve it exactly with the Hungarian method [13] as implemented in SciPy [16], separately for every layer and tier; cost matrices have at most 614×614 entries (all components of a feed-forward layer). We apply it to the magnitude hybrid, keeping its SLC set and choosing the slot of each component within its tier; this is our *fault-aware mapping*. As a high-cost reference, we apply the same mapping to an all-SLC chip.

E. Joint Tier and Slot Mapping

Fault-aware mapping keeps the SLC set fixed. A natural extension chooses tier and slot together for each chip: an important component might be better off in a clean MLC slot than in a badly damaged SLC slot. We form one assignment over all k components and all k slots of a layer, with $\lceil fk \rceil$ SLC slots, which enforces the SLC budget. The cost of a slot adds the expected damage from programming noise in its tier to Eq. (13),

$$C_{rs}^{\text{joint}} = C_{rs} + \kappa_{\tau(s)} \Big(s_r^2 \sum_j F\big(\sigma_r v_{rj}\big) + \sum_i t_i^2 F\big(u_{ir}\big) \Big), \tag{14}$$

where $\kappa_\tau = 2(\sigma_d/\Delta_\tau)^2 \sum_t L_\tau^{2t}$ is the noise variance of a weight in tier τ, in units of its 8-bit step. The second term is the Fisher score D_r of Eq. (10) scaled by the tier. We use $\sigma_d = \max(\sigma, 2\%)$, so that on a chip with stuck cells but no programming noise the tier choice still reflects the higher fragility of MLC.

F. Checking the Cost Model

Before every run, a self-test programs a random layer with a known fault map and compares the predicted cost $\sum_r C_{rr}$ with the actual Fisher-weighted error of the programmed crossbar. The run stops unless the two agree to within 10% for both SLC and MLC, and unless the assignment does not increase the cost.

VI. Noise- and Fault-Aware Training

Mapping leaves the weights unchanged. Training instead changes them so that the model tolerates the errors it will meet.

A. Training on Simulated Chips

Starting from the fine-tuned SVD model of Section IV, every training step draws a new chip. Each factor of every layer is programmed with the crossbar model of Section III, using the same code as the evaluation, and the forward pass uses the stored values $\hat{W}$ in place of the clean weights. The chip model is not differentiable, so gradients pass through it unchanged, a straight-through estimator [12]:

$$W_{\text{fwd}} = W + \text{sg}\left(\hat{W} - W\right), \qquad \frac{\partial L}{\partial W} = \frac{\partial L}{\partial W_{\text{fwd}}}, \tag{15}$$

where sg stops the gradient. Because a new chip is drawn at every step, the model cannot adapt to any single fault map. The training chip has programming noise $\sigma = 5\%$ and, where stated, 0.5% stuck cells. Training runs for 2 epochs over the SST-2 training set with AdamW, learning rate 10^{-5} with linear decay and batch size 32. Dropout stays active as in normal fine-tuning.

B. Variants

The variants differ only in the chip simulated during training. Every trained model is then tested on the same hybrid chip, with 20% of the components in SLC.

- **Noise only**: every component on SLC cells, programming noise only. This is plain noise-aware training.
- **Noise + faults**: every component on SLC cells, with noise and stuck cells.
- **Tier-aware**: the chip the model will run on. The top 20% of components of each layer by $|\sigma_r|$ use SLC cells and the rest MLC cells, with noise and stuck cells. Because singular values change during training, the SLC set is recomputed from the current values every 200 steps.

We call a variant *tier-aware* when it knows the tier, SLC or MLC, in which each component will be stored, and trains every component under the errors of its own tier. The first two variants are tier-unaware: every component sees the same cells, as in standard hardware-aware training [11]. The tier-aware variant applies MLC-level errors exactly where MLC will be used, so the components in the fragile tier learn to tolerate its errors. Comparing the variants in turn isolates the effect of injecting stuck cells and of knowing the tiers.

The tier-unaware variants use SLC cells for a practical reason. A first attempt placed every component on MLC cells,

TABLE II
EXPERIMENTS. CHIPS: NUMBER OF SIMULATED CHIPS PER POINT.

Experiment	Swept	Fixed	Chips
Programming noise	$\sigma = 0$–10%		5
SLC share	share 5–50%	$\sigma = 3, 5, 7.5\%$	20
Stuck cells	$p = 0.1$–5%		20
Read noise	$\sigma_r = 0.5$–5%		5
Level noise	mid levels $\times 1.5$, $\times 2$	$\sigma = 5\%$	10
Fault pattern	scattered, clusters, lines	$p = 1\%$	10
IR drop	$r = 10^{-4}$–3×10^{-3}		1
ADC resolution	3–8 bits		1
ADC + noise*	4–8 bits	$\sigma = 5\%$	3
Fault-aware mapping	$p = 0.2$–2%		10
	$p = 0.2$–1%	$\sigma = 5\%$	10
	clusters, lines	$p = 1\%$	10
Training	as mapping; $\sigma = 0$–7.5%		10

*DistilBERT only. ADC experiments use bit-serial inputs.

the tier-unaware choice that matches the cheaper tier. At 5% noise and 0.5% stuck cells such a chip is at chance accuracy, and the training loss stayed at about 0.70, close to $\ln 2$: the model learned to hedge between the two classes rather than to tolerate the errors. On SLC cells the loss fell well below this level from the first steps.

VII. EXPERIMENTAL SETUP

Models and data. We use two publicly available SST-2 classifiers from the Hugging Face hub, BERT-Base with 12 encoder layers[1] and DistilBERT with 6 encoder layers[2]. All encoder linear layers are factorised and mapped to crossbars; embeddings, layer normalisation, attention score computation and the classification head (including BERT's pooler) stay digital. Accuracy is measured on all 872 sentences of the SST-2 validation set, with inputs truncated to 128 tokens. Placement scores and Fisher information use 2,048 sentences from the SST-2 training set, the same sentences for every score.

Section IX repeats part of the study on MNLI with its own setup.

Experiments. Table II lists the experiments. Hybrid placements keep 20% of the components in SLC unless the SLC share is the swept quantity. Each error source is varied on its own, with all others disabled, so that its effect can be attributed. Where a configuration contains no random error, such as IR drop or ADC resolution without noise, a single evaluation is exact and only one chip is used.

Evaluation batches. Inputs to every crossbar are quantised to 8 bits with one scale per tensor, and a tensor holds a whole evaluation batch, so accuracy depends slightly on how sentences are grouped: without noise it shifts by up to about one point between batch sizes. Within an experiment every placement uses the same batches, so comparisons are unaffected. The basic sweeps (programming noise, SLC share, stuck cells) and the mapping and training experiments use batches of 128; the experiments on the other non-idealities (read noise to ADC) use batches of 64.

[1]textattack/bert-base-uncased-SST-2

[2]distilbert-base-uncased-finetuned-sst-2-english

TABLE III
SST-2 ACCURACY (%) WITHOUT HARDWARE ERRORS.

Model	FP32	INT8	SVD	SVD, fine-tuned
BERT-Base	92.43	91.86	89.33	90.14
DistilBERT	91.06	90.37	89.68	89.11

Statistics. We report the mean accuracy over chips and, where useful, the worst chip. Because placements share chips (Section III), we compare two placements by their paired difference: the number of correctly classified sentences, out of 872, by which one placement beats the other on the same chip, or the same difference in accuracy points, averaged over chips and given with its standard error.

Compute. All runs use Kaggle notebooks with one NVIDIA T4 GPU, PyTorch and Hugging Face Transformers. Each stage is a single self-contained script: factorisation and fine-tuning, the placement experiments, fault-aware mapping, and training. Every script runs its self-tests before any experiment and stops if one fails.

VIII. RESULTS

We report BERT-Base in the main text; DistilBERT behaves the same way unless stated, and its tables are in Section A. The two central comparisons, the paired comparison of placements (Table V) and fault-aware mapping (Table VII), show both models, because their conclusions rest on both. Accuracies are means over chips, and hybrid placements keep 20% of the components in SLC unless stated otherwise.

A. Cost of SVD and the Error-Free Chip

Table III gives the accuracy before any hardware error. Post-training 8-bit quantisation costs 0.6–0.7 points. The SVD factorisation costs about two points even after fine-tuning; for DistilBERT, fine-tuning slightly lowered accuracy, which suggests that 8 epochs over-fit. This cost is paid once and is the same for every placement.

On a chip with no errors, every placement reaches 89.1–90.0% on BERT-Base and 88.6–89.8% on DistilBERT, within about one point of the digital SVD model. The remaining differences come from quantisation alone: each tier's crossbars have their own 8-bit scales, and the intermediate vector of each layer is re-quantised to 8 bits, so the rounding depends on how the components are split between the tiers.

B. Programming Noise

Figure 2 shows accuracy against programming noise. All-MLC storage fails first, falling to 66.3% at $\sigma = 5\%$ and close to chance at 7.5%. Keeping 20% of the components in SLC by magnitude or Fisher stays within about a point of all-SLC up to $\sigma = 5\%$. The gradient placement keeps up until 5% but falls behind at higher noise (79.3% against 83.5% for magnitude at 7.5%), and a random 20% recovers little: *which* components are protected matters more than how many.

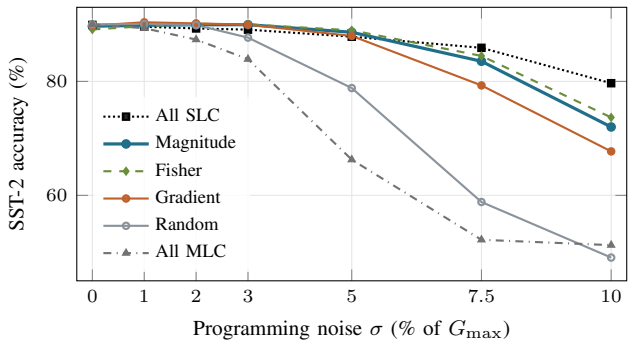


Fig. 2. BERT-Base accuracy against programming noise, mean of 5 chips. Hybrids keep 20% of the components in SLC.

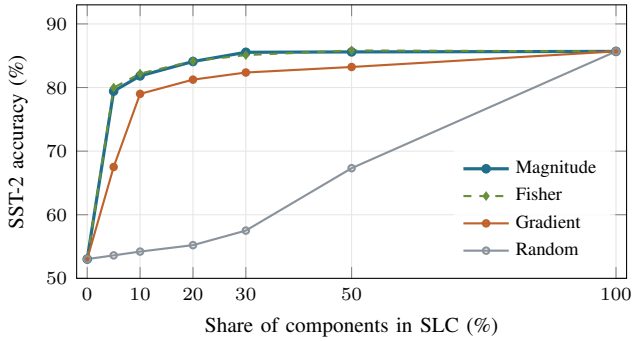


Fig. 3. BERT-Base accuracy against the share of components in SLC at $\sigma = 7.5\%$, mean of 20 chips. 0% is all-MLC and 100% all-SLC.

C. Choosing and Sizing the Protected Set

Figure 3 and Table IV vary the share of components in SLC, with 20 chips per point. Magnitude and Fisher lie on top of each other at every share. The gradient is close to them at $\sigma = 3\%$, but at 7.5% it trails by 12 points with 5% SLC and by 2–3 points at larger shares. Random selection approaches the others only as the share grows towards one half.

Paired comparison. Because all placements see the same chips, Table V compares them chip by chip against magnitude. Over all 300 chips of the study (3 noise levels, 5 SLC shares, 20 chips), Fisher differs from magnitude by $+0.4 \pm 0.7$ sentences on BERT-Base and -0.6 ± 0.7 on DistilBERT, out of 872: no measurable difference at any noise level. The gradient is behind by 17.3 ± 2.0 and 7.8 ± 1.1 sentences, and the gap grows with noise, to 40.4 ± 5.0 sentences on BERT-Base at $\sigma = 7.5\%$. Section X explains why.

How much SLC is needed. With magnitude ranking, the smallest share that comes within half a point of all-SLC is 5% at $\sigma = 3\%$ for both models, 5% (BERT-Base) to 10% (DistilBERT) at $\sigma = 5\%$, and 30% (BERT-Base) to 50% (DistilBERT) at $\sigma = 7.5\%$. In cells per weight (Section IV), 10% SLC needs 61% and 30% SLC 70% of the cells of an all-SLC chip. The share should therefore follow the expected noise: a small share suffices at moderate noise, and the saving shrinks as noise grows.

TABLE IV
BERT-BASE ACCURACY (%) BY SHARE OF COMPONENTS IN SLC, MEAN OF 20 CHIPS. 0% AND 100% ARE ALL-MLC AND ALL-SLC.

σ	Placement	0%	5%	10%	20%	30%	50%	100%
3%	Magnitude	82.1	89.7	89.9	89.8	89.8	89.7	89.2
	Fisher		89.6	89.8	89.8	89.8	89.8	
	Gradient		88.7	90.0	90.0	89.9	89.6	
	Random		87.0	86.9	87.6	87.3	89.0	
5%	Magnitude	66.8	88.0	88.6	88.7	88.9	88.9	88.0
	Fisher		87.9	88.6	88.8	88.9	88.9	
	Gradient		84.2	88.2	88.4	88.4	88.1	
	Random		72.5	73.6	76.2	77.8	83.4	
7.5%	Magnitude	53.0	79.4	81.8	84.1	85.5	85.6	85.7
	Fisher		80.0	82.2	84.2	85.1	85.8	
	Gradient		67.5	79.0	81.2	82.4	83.2	
	Random		53.6	54.2	55.2	57.5	67.3	

TABLE V
PAIRED DIFFERENCE TO MAGNITUDE, IN CORRECTLY CLASSIFIED SENTENCES OUT OF 872 (MEAN ± STANDARD ERROR OVER 100 CHIPS PER NOISE LEVEL, 300 OVERALL).

Model	Placement	$\sigma = 3\%$	5%	7.5%	All
BERT-Base	Fisher	-0.1 ± 0.5	-0.1 ± 0.7	$+1.4 \pm 1.9$	$+0.4 \pm 0.7$
	Gradient	-1.2 ± 0.7	-10.4 ± 1.6	-40.4 ± 5.0	-17.3 ± 2.0
	Random	-19.4 ± 1.4	-104 ± 6	-224 ± 6	-116 ± 6
DistilBERT	Fisher	-0.4 ± 0.7	-1.3 ± 0.7	$+0.0 \pm 1.7$	-0.6 ± 0.7
	Gradient	-0.9 ± 0.7	-4.0 ± 1.0	-18.4 ± 2.6	-7.8 ± 1.1
	Random	-11.2 ± 1.1	-66 ± 5	-186 ± 7	-88 ± 5

D. Stuck Cells

Figure 4 makes a share p of all cells stuck, half at $G_{\max}$ and half at $G_{\min}$, with no programming noise. Stuck cells are far more harmful than noise, because a stuck cell in a high-order slice shifts its weight by a large fixed amount. All-MLC fails when one cell in a thousand is stuck. Up to $p = 0.2\%$, magnitude and Fisher with 20% SLC match or exceed all-SLC, but from 0.5% every placement degrades quickly, and at 1% even all-SLC loses 14–27 points across the two models. Survival then depends on *where* the stuck cells fall: at 0.5% the worst of 20 BERT-Base chips reaches only 66.2% with magnitude placement. No placement solves this, which motivates fault-aware mapping.

E. Other Non-Idealities

Table VI adds the remaining error sources one at a time. These experiments use evaluation batches of 64, so their error-free row differs slightly from Fig. 2.

Read and level-dependent noise behave like programming noise. All-MLC fails (53.0% at 5% read noise), magnitude and Fisher stay at or above all-SLC on BERT-Base (about one point below on DistilBERT at 5% read noise), and the gradient trails them by six points at 5% read noise. Making the middle MLC levels twice as noisy costs all-MLC a further five points but the hybrids only about half a point, since their most important components do not use MLC at all.

Fault patterns. At 1% stuck cells, clustered faults harm all-SLC less than scattered ones, because a cluster damages fewer

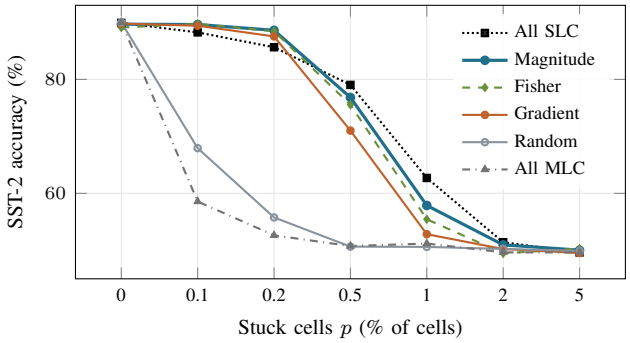


Fig. 4. BERT-Base accuracy against the share of stuck cells, no programming noise, mean of 20 chips. The horizontal axis is not to scale.

TABLE VI
BERT-BASE ACCURACY (%) UNDER EACH OF THE OTHER NON-IDEALITIES. HYBRIDS KEEP 20% OF THE COMPONENTS IN SLC.

Condition	All SLC	All MLC	Magn.	Fisher	Grad.
No errors	89.7	90.4	90.6	89.6	90.3
Read noise $\sigma_r = 2\%$	89.0	77.3	90.1	89.7	89.8
Read noise $\sigma_r = 5\%$	80.0	53.0	82.5	82.7	76.6
$\sigma = 5\%$, uniform levels	88.2	67.3	88.9	89.0	88.3
$\sigma = 5\%$, middle levels $\times 2$	88.2	62.5	88.4	88.4	87.8
1% stuck, scattered	61.3	50.7	56.4	55.6	52.6
1% stuck, 3×3 clusters	66.7	50.8	54.0	53.1	51.5
1% stuck, bit lines	57.0	50.4	50.4	52.2	51.0
ADC 7 bits	90.0	89.2	89.3	89.7	90.1
ADC 6 bits	89.3	83.0	90.0	90.5	89.6
ADC 5 bits	51.1	51.1	51.1	51.6	53.0
IR drop $r = 3 \times 10^{-4}$	88.9	89.8	90.4	91.1	90.5
IR drop $r = 10^{-3}$	87.0	90.6	88.1	88.3	88.6
IR drop $r = 3 \times 10^{-3}$	58.5	79.1	76.8	72.8	76.4

distinct weights; whole broken bit lines are the most harmful pattern. The hybrids are already close to chance at this rate for every pattern, in line with Fig. 4.

ADC resolution. With the ADC range fixed by calibration, 6 bits suffice for all-SLC and for every hybrid, while all-MLC needs 7. At 5 bits BERT-Base fails for every placement, and DistilBERT drops to 71–77% (56.5% for all-MLC). With 5% programming noise (DistilBERT, 3 chips), a 6-bit ADC keeps the magnitude hybrid at 88.5%, against 56.0% for all-MLC. Since ADCs dominate the energy of crossbar accelerators [1], this resolution is the natural target for a later cost study.

IR drop. At $r \leq 3 \times 10^{-4}$ no placement loses more than a point; differences of this size come from a single noise-free evaluation and are not significant. At $r = 3 \times 10^{-3}$ IR drop hurts SLC most: all-SLC falls to 58.5% while all-MLC keeps 79.1%. An SLC cell storing a one sits at $G_{\max}$, and by Eq. (6) the highest conductances lose the largest share of their current.

F. Fault-Aware Mapping

Table VII follows the stuck-cell results step by step, on 10 chips per point. For each model, the first two rows are the high-cost all-SLC chip, without and with fault-aware mapping; the remaining rows are the hybrid chip, first with no mitigation, then with fault-aware mapping, and then with each training variant added. We compare the hybrid against the all-SLC chip *with* mapping, since mapping costs no hardware and would be applied to any chip. Every row sees the same chips, since the trained models keep the layer shapes; a re-run of the untrained model at 0.5% stuck cells reproduced all 40 chip results exactly. Differences quoted below are paired over these chips, as mean $\pm$ standard error in accuracy points.

Mapping helps in every setting. Fault-aware mapping raises the hybrid at every fault rate and pattern, for both models. At 0.5% stuck cells BERT-Base rises from 76.6% to 85.6% and DistilBERT from 77.9% to 85.1%, and at 0.2% the loss almost disappears. Chips also become more alike: at 0.5% the standard deviation over BERT-Base chips falls from 6.7 to 1.2 points. The largest gains come with clustered and line faults (up to 13.9 ± 3.3 points), where moving important components away from a damaged region matters most.

Choosing the tier per chip does not help. Joint tier and slot mapping (Section V) wins in some settings and loses in others, by -5.3 ± 2.4 to $+7.0 \pm 2.3$ points against fault-aware mapping, and most differences are within two standard errors of zero. We attribute this to the cost model: its second-order estimate underrates the large error of a stuck high-order cell, so it sometimes moves an important component into MLC for a small modelled saving. The benefit of mapping comes from the choice of slot, not of tier.

Mapping alone is not enough. The mapped hybrid matches all-SLC with mapping up to 0.2% stuck cells, but falls behind from 0.5% upwards: at 1% it reaches 65.1% against 75.8% on BERT-Base. In an all-SLC chip every component sits in the tier where a stuck cell does least damage, and the mapping has five times as many SLC slots to choose from.

G. Noise- and Fault-Aware Training

The last three rows of Table VII add training to fault-aware mapping, and Table VIII tests the trained models under programming noise alone.

Training must include stuck cells. Training with programming noise alone gives no consistent gain under stuck cells: at 1% it reaches 67.6% and 68.2%, against 65.1% and 67.0% with mapping alone. Adding stuck cells to the training chip raises accuracy at 1% to 78.6% and 80.7%, a gain of 13.5 ± 3.1 and 13.7 ± 2.6 points.

Training each tier on its own cells helps most. Tier-aware training raises accuracy at 1% stuck cells further, to 82.9% on BERT-Base and 84.7% on DistilBERT ($+4.3 \pm 2.2$ and $+4.0 \pm 0.6$ points over tier-unaware training), and to 80.5% on BERT-Base when programming noise is added. Its advantage grows with the density of errors: at 0.5% the two training variants are equal, and at 0.2% tier-aware training is slightly behind. This is the price of training on the harsher hybrid chip, which costs 0.3 and 1.3 points on an error-free chip (Table VIII).

Training and mapping are complementary. Training makes the weights tolerate errors, while mapping keeps the most sensitive values away from known faults. Each helps

TABLE VII

ACCURACY (%) UNDER STUCK CELLS, MEAN OF 10 CHIPS (BEST PER COLUMN IN BOLD). THE ALL-SLC CHIP STORES EVERY COMPONENT IN SLC AND IS THE HIGH-COST REFERENCE; THE HYBRID CHIP KEEPS 20% OF THE COMPONENTS IN SLC, CHOSEN BY MAGNITUDE, AND USES 34% FEWER CELLS. THE CHIP COLUMN IS THE CHIP EACH MODEL IS TESTED ON; THE TRAINING VARIANTS (SECTION VI) DIFFER ONLY IN THE CHIP SIMULATED DURING TRAINING.

		Stuck cells, no noise				Stuck cells, $\sigma = 5\%$			1% stuck cells	
Chip	Mitigation	0.2%	0.5%	1%	2%	0.2%	0.5%	1%	clusters	lines
BERT-Base										
All SLC	None	85.4	80.1	60.9	51.1	85.6	78.4	64.6	66.5	57.2
	Fault-aware mapping	89.6	83.7	75.8	**55.9**	86.1	82.4	76.5	81.3	77.5
Hybrid, 20% SLC	None	88.5	76.6	55.9	50.1	84.9	70.3	52.1	53.7	50.1
	Fault-aware mapping	**90.1**	85.6	65.1	52.2	87.4	80.4	58.1	64.3	56.9
	Noise-only training + fault-aware mapping	89.8	87.0	67.6	51.7	88.0	82.7	58.0	65.8	58.9
	Noise + fault training + fault-aware mapping	89.9	**88.7**	78.6	50.8	**89.7**	87.4	72.9	78.8	72.6
	Tier-aware training + fault-aware mapping	88.8	88.3	**82.9**	53.0	89.1	**87.8**	**80.5**	**85.0**	**77.7**
DistilBERT										
All SLC	None	88.1	85.4	73.7	57.9	86.9	81.7	68.0	79.6	71.1
	Fault-aware mapping	88.0	87.6	**84.8**	**62.3**	87.6	86.8	**80.7**	**85.6**	81.5
Hybrid, 20% SLC	None	87.1	77.9	62.4	50.2	85.4	74.9	59.5	55.9	54.0
	Fault-aware mapping	88.4	85.1	67.0	53.3	87.7	83.0	65.1	68.5	68.0
	Noise-only training + fault-aware mapping	87.3	84.5	68.2	51.5	85.9	82.8	59.9	66.1	65.9
	Noise + fault training + fault-aware mapping	**88.6**	**88.2**	80.7	55.0	**88.8**	86.4	77.5	80.7	81.6
	Tier-aware training + fault-aware mapping	88.2	88.1	84.7	57.4	88.2	**86.9**	79.8	83.4	**84.4**

TABLE VIII

ACCURACY (%) UNDER PROGRAMMING NOISE ALONE, WITHOUT STUCK CELLS, MEAN OF 10 CHIPS. ALL HYBRIDS KEEP 20% OF THE COMPONENTS IN SLC, CHOSEN BY MAGNITUDE; $\sigma = 0$ IS AN ERROR-FREE CHIP.

	BERT-Base			DistilBERT		
Training	$\sigma = 0$	5%	7.5%	0	5%	7.5%
No training	89.7	88.7	84.2	89.1	88.2	85.6
Noise-only training	89.9	89.7	86.3	87.4	87.6	85.6
Noise + fault training	**90.9**	**89.9**	**88.8**	**89.9**	**89.3**	87.6
Tier-aware training	89.4	89.3	88.3	87.8	88.4	**88.0**

without the other: tier-aware training without mapping reaches 76.1% and 81.3% at 1% stuck cells, above mapping alone, and adding mapping still gains 6.8 ± 2.2 and 3.5 ± 1.2 points.

The gap to all-SLC closes. With tier-aware training and mapping, the hybrid beats all-SLC with mapping on BERT-Base at 0.5% and 1% stuck cells (88.3% against 83.7%, and 82.9% against 75.8%) and with programming noise. On DistilBERT it matches all-SLC within one point up to 1%, is ahead for broken bit lines and two points behind for clusters. The hybrid does this with 34% fewer cells. At 2% stuck cells every configuration still fails.

Training also helps against programming noise. Without stuck cells (Table VIII), noise + fault training raises the hybrid at $\sigma = 7.5\%$ from 84.2% to 88.8% on BERT-Base, and tier-aware training from 85.6% to 88.0% on DistilBERT. Noise-only training helps BERT-Base (86.3%) but not DistilBERT. On an error-free chip, noise + fault training even gains about a point.

TABLE IX

MNLI ACCURACY (%) ON 2,000 MATCHED VALIDATION PAIRS (CHANCE 33.3%). HYBRIDS KEEP 20% OF THE COMPONENTS IN SLC; PAIRED DIFFERENCES ARE IN POINTS OVER ALL SLC SHARES (100 CHIPS).

		BERT-Base	DistilBERT
Digital SVD model		83.2	80.9
Error-free chip	All SLC	72.2	76.9
	Magnitude	79.1	77.3
$\sigma = 5\%$	Magnitude	**68.0**	**71.5**
	Fisher	67.8	71.1
	Gradient	63.3	71.3
	Random	35.9	44.1
Fisher minus magnitude		-0.2 ± 0.2	-0.2 ± 0.1
Gradient minus magnitude		-3.4 ± 0.4	-0.5 ± 0.2
0.5% stuck	Magnitude	40.0	49.1
	+ fault-aware mapping	**60.0**	**67.9**
	+ joint mapping	60.4	67.0

IX. GENERALISATION TO MNLI

To test whether the placement and mapping results depend on SST-2, we repeated part of the study on MNLI, a three-class natural language inference task [7], [17]. We used public BERT-Base[3] and DistilBERT[4] checkpoints with the same simulator, scores and mapping code. Each model was factorised and fine-tuned for 2 epochs on 100,000 MNLI training pairs, then evaluated on a fixed subset of 2,000 matched validation pairs. We ran the SLC-share study at $\sigma = 5\%$ (20 chips), the stuck-cell sweep (20 chips) and fault-aware mapping (10 chips); training, other noise levels and the remaining error sources were not run. Table IX summarises the results.

[3]textattack/bert-base-uncased-MNLI

[4]typeform/distilbert-base-uncased-mnli

The placement results hold. Fisher ranks the components almost as magnitude does (rank correlation 0.96–0.97, against 0.56–0.57 for the gradient) and reaches the same accuracy on the same chips, within a fifth of a point. On BERT-Base the gradient is 3.4 points behind; on DistilBERT the three scores are close, and the gradient is ahead only at 50% SLC. A random 20% recovers little.

The mapping results hold. MNLI is more sensitive to stuck cells than SST-2: all-MLC falls to 37–45% at 0.1% stuck cells, and the magnitude hybrid to 40–50% at 0.5%. Fault-aware mapping recovers much of this, by 20.1 ± 3.0 points on BERT-Base and 18.8 ± 3.2 on DistilBERT at 0.5%. Choosing the tier per chip again adds nothing (within one point of fault-aware mapping).

A limitation of the simulator. On MNLI the error-free chip loses accuracy against the digital model, by up to 11 points for BERT-Base, and the loss depends on the placement. The cause is the vector between the two crossbars of a tier, which holds one value per component and is re-quantised to 8 bits with a single scale. The largest components set that scale, so the outputs of small components are rounded away. A single tier, as in all-SLC or all-MLC, suffers most (72.2% and 72.4% on BERT-Base); a random split helps little (73.0%); separating large from small components by magnitude gives each tier its own scale and recovers most of the loss (79.1%). On SST-2 the same effect stays within about a point. Because of it, MNLI comparisons with all-SLC or all-MLC are not meaningful, and we compare only placements and mappings with the same tier sizes, which share this quantisation.

X. Discussion

A. Why Magnitude Ranking Works

The cell model explains why ranking by singular value matches the Fisher estimate. Each crossbar column is quantised with its own scale (Eq. (2)), and the column of component r in the first crossbar holds $\sigma_r v_r$, so its scale is $s_r = |\sigma_r| \, \|v_r\|_\infty / 127$. A fixed cell noise therefore causes a weight error proportional to $|\sigma_r|$: components with large singular values receive the largest absolute errors. The Fisher score (Eq. (10)) multiplies this error size by the loss sensitivity of each stored value. That it selects essentially the same components as magnitude suggests that the error size varies across components much more than the sensitivity. The gradient score measures sensitivity to a small change of σ_r alone, ignoring both the error size and the errors on u_r and v_r, and so falls behind when errors are large.

B. Relation to HyFlexPIM

Our results agree with HyFlexPIM [3] that a hybrid SLC/MLC layout of SVD components works and that all-MLC storage does not, but not on how to rank components. HyFlexPIM compares its gradient ranking with magnitude- and rank-based alternatives on MRPC and CoLA at one noise level and reports differences of about a point or less in favour of the gradient. We find no difference at mild noise ($\sigma = 3\%$) and the reverse at high noise, so the findings need not conflict. Three differences in set-up explain this:

- **Noise model.** Under HyFlexPIM's relative weight noise the error of a weight does not depend on the scale of its crossbar column, so the mechanism above is absent. Its noise level is set from a bit error rate measured on real MLC cells [8], which is itself a per-cell quantity.
- **Noise range.** HyFlexPIM evaluates a single noise level. In our sweeps the rankings differ only at $\sigma \geq 5\%$.
- **When gradients are taken.** HyFlexPIM records gradients during its 1–3 epochs of fine-tuning, while ours are taken on the fine-tuned model. We did not repeat their procedure, since magnitude already matches the Fisher estimate.

C. Limitations

- **One error source at a time.** Each non-ideality was varied on its own; their combined effect on one chip is not measured. The IR-drop model is static, and the ADC range is set by the largest calibration value rather than a percentile.
- **Parameter ranges.** Noise levels, stuck-cell rates and wire resistance were swept from harmless to failure, not calibrated to a particular device, so we report trends and comparisons between placements rather than absolute accuracy of a specific chip.
- **Known faults.** Fault-aware mapping requires each chip to be tested and mapped once before programming. Cells that fail later, in use, require testing and remapping again. We did not measure the time this per-chip mapping takes.
- **Cost.** Savings are given in memory cells only; energy, latency and area of the peripheral circuits, in particular the ADCs, are not evaluated.
- **Activation quantisation.** Crossbar inputs are quantised with one scale per tensor. This shifts SST-2 accuracy by up to about a point between batch sizes, and on MNLI it costs up to 11 points even on an error-free chip, because the vector between a tier's two crossbars shares one scale across all its components (Section IX). All comparisons use identical batches, but finer scales, per token or per group of components, are needed before absolute accuracies on harder tasks can be trusted.
- **Training settings.** Each training variant was run once, with one training chip ($\sigma = 5\%$, 0.5% stuck cells) and 2 epochs; other settings may trade error-free accuracy against robustness differently.
- **Scope.** The full study is on SST-2 with two models; on MNLI only placement and mapping were run, without training or the other non-idealities. The SVD step itself costs about two points, and DistilBERT's fine-tuning would benefit from early stopping.

XI. Conclusion and Future Work

A hybrid SLC/MLC layout is necessary for SVD-factorised transformers on ReRAM, and which components are protected matters as much as how many. Ranking by singular value is

close to the best placement under a cell-level error model and needs no data; a small SLC share then suffices at moderate noise, and the hybrid stays robust to read noise, level-dependent noise, a 6-bit ADC and moderate IR drop. Stuck cells are the exception. Mapping components to slots around a tested chip's faults recovers most of the loss at no hardware cost, and tier-aware training closes the remaining gap to an all-SLC chip up to 1% scattered stuck cells, with a third fewer cells.

Several directions remain open. The first is an evaluation of all error sources together on one chip, combining placement, mapping and training, and a study of the training settings, of which we tested one. The second is the hardware cost of each configuration, energy, latency and area including the ADCs, with a simulator such as NeuroSim [18]. The third is security: crossbar-stored weights are exposed to fault-injection attacks such as the bit-flip attack [19], and multi-level cells may lower the cost of such attacks. The simulator also needs finer activation scales, per token or per group of components, which MNLI showed to matter on harder tasks. Finally, the full pipeline, including training, can be applied to MNLI and other GLUE tasks such as MRPC, to convolutional networks, and to an FPGA emulation of the crossbar.

REFERENCES

[1] A. Shafiee, A. Nag, N. Muralimanohar, R. Balasubramonian, J. P. Strachan, M. Hu, R. S. Williams, and V. Srikumar, "ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars," in *Proc. 43rd Annu. Int. Symp. Comput. Archit. (ISCA)*, 2016, pp. 14–26.

[2] P. Chi, S. Li, C. Xu, T. Zhang, J. Zhao, Y. Liu, Y. Wang, and Y. Xie, "PRIME: A novel processing-in-memory architecture for neural network computation in ReRAM-based main memory," in *Proc. 43rd Annu. Int. Symp. Comput. Archit. (ISCA)*, 2016, pp. 27–39.

[3] C. E. Song, P. Bhatnagar, Z. Xia, N. S. Kim, T. S. Rosing, and M. Kang, "Hybrid SLC-MLC RRAM mixed-signal processing-in-memory architecture for transformer acceleration via gradient redistribution," in *Proc. 52nd Annu. Int. Symp. Comput. Archit. (ISCA)*, 2025.

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[5] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," 2019.

[6] R. Socher, A. Perelygin, J. Wu, J. Chuang, C. D. Manning, A. Ng, and C. Potts, "Recursive deep models for semantic compositionality over a sentiment treebank," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2013, pp. 1631–1642.

[7] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "GLUE: A multi-task benchmark and analysis platform for natural language understanding," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.

[8] K. Fan, W.-C. Chen, S. Pinge, H.-S. P. Wong, and T. Rosing, "Efficient open modification spectral library searching in high-dimensional space with multi-level-cell memory," 2024.

[9] L. Chen, J. Li, Y. Chen, Q. Deng, J. Shen, X. Liang, and L. Jiang, "Accelerator-friendly neural-network training: Learning variations and defects in RRAM crossbar," in *Proc. Design, Automation & Test in Europe (DATE)*, 2017, pp. 19–24.

[10] L. Xia, M. Liu, X. Ning, K. Chakrabarty, and Y. Wang, "Fault-tolerant training with on-line fault detection for RRAM-based neural computing systems," in *Proc. 54th Design Automation Conf. (DAC)*, 2017.

[11] M. J. Rasch *et al.*, "Hardware-aware training for large-scale and diverse deep learning inference workloads using in-memory computing-based accelerators," *Nature Communications*, vol. 14, p. 5282, 2023.

[12] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," 2013.

[13] H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955.

[14] Y. LeCun, J. S. Denker, and S. A. Solla, "Optimal brain damage," in *Advances in Neural Information Processing Systems (NIPS)*, vol. 2, 1990.

[15] J. Kirkpatrick *et al.*, "Overcoming catastrophic forgetting in neural networks," *Proc. Natl. Acad. Sci. USA*, vol. 114, no. 13, pp. 3521–3526, 2017.

[16] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[17] A. Williams, N. Nangia, and S. R. Bowman, "A broad-coverage challenge corpus for sentence understanding through inference," in *Proc. NAACL-HLT*, 2018, pp. 1112–1122.

[18] X. Peng, S. Huang, Y. Luo, X. Sun, and S. Yu, "DNN+NeuroSim: An end-to-end benchmarking framework for compute-in-memory accelerators with versatile device technologies," in *Proc. IEEE Int. Electron Devices Meeting (IEDM)*, 2019.

[19] A. S. Rakin, Z. He, and D. Fan, "Bit-flip attack: Crushing neural network with progressive bit search," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 1211–1220.

APPENDIX A
DISTILBERT RESULTS

Tables X to XIII repeat the experiments of Section VIII for DistilBERT, with the same chips per point. Fault-aware mapping results for both models are in Table VII.

TABLE X
DISTILBERT ACCURACY (%) AGAINST PROGRAMMING NOISE σ, MEAN OF 5 CHIPS.

Placement	0	1%	2%	3%	5%	7.5%	10%
All SLC	89.0	88.7	88.8	88.6	88.5	87.5	85.9
All MLC	89.1	88.4	87.9	86.1	74.5	60.4	56.0
Magnitude	89.1	88.9	88.9	88.9	88.4	85.9	77.7
Fisher	89.0	89.0	89.0	89.1	88.5	85.5	77.6
Gradient	89.8	88.5	88.9	88.7	87.7	83.4	69.8
Random	88.6	88.7	88.7	87.8	82.8	61.8	52.4

TABLE XI
DISTILBERT ACCURACY (%) AGAINST THE SHARE OF STUCK CELLS p, NO PROGRAMMING NOISE, MEAN OF 20 CHIPS.

Placement	0	0.1%	0.2%	0.5%	1%	2%	5%
All SLC	89.0	88.2	87.8	85.2	74.6	58.5	51.5
All MLC	89.1	65.5	58.2	52.3	51.1	50.2	49.4
Magnitude	89.1	88.3	87.4	80.3	62.1	51.4	49.9
Fisher	89.0	88.3	87.2	80.3	62.1	51.3	50.4
Gradient	89.8	88.1	86.7	74.5	55.4	51.0	50.5
Random	88.6	76.9	63.3	51.3	51.0	49.4	50.1

TABLE XII
DISTILBERT ACCURACY (%) BY SHARE OF COMPONENTS IN SLC, MEAN OF 20 CHIPS. 0% AND 100% ARE ALL-MLC AND ALL-SLC.

σ	Placement	0%	5%	10%	20%	30%	50%	100%
3%	Magnitude	86.0	88.4	88.6	88.8	88.9	88.7	88.7
	Fisher		88.5	88.7	88.7	88.8	88.6	
	Gradient		88.4	88.5	88.6	88.8	88.7	
	Random		86.4	86.8	87.5	88.1	88.4	
5%	Magnitude	74.8	87.4	88.0	88.1	88.8	88.4	88.2
	Fisher		87.3	87.8	88.4	88.1	88.3	
	Gradient		87.3	87.3	87.7	88.1	87.8	
	Random		75.4	77.4	81.9	82.3	85.7	
7.5%	Magnitude	56.9	80.2	84.0	85.2	86.6	86.9	87.4
	Fisher		80.8	83.8	85.4	86.0	87.0	
	Gradient		78.0	80.7	83.1	84.8	86.0	
	Random		56.4	58.2	63.1	64.7	73.7	

TABLE XIII
DISTILBERT ACCURACY (%) UNDER EACH OF THE OTHER NON-IDEALITIES. HYBRIDS KEEP 20% OF THE COMPONENTS IN SLC.

Condition	All SLC	All MLC	Magn.	Fisher	Grad.
No errors	88.5	89.0	88.8	89.1	88.6
Read noise $\sigma_r = 2\%$	88.3	83.8	88.4	88.2	88.1
Read noise $\sigma_r = 5\%$	84.5	57.7	83.3	83.2	80.5
$\sigma = 5\%$, uniform levels	88.4	75.7	88.4	88.1	88.0
$\sigma = 5\%$, middle levels $\times 2$	88.4	70.3	88.0	87.9	87.1
1% stuck, scattered	74.2	51.2	62.5	60.0	55.0
1% stuck, 3×3 clusters	80.3	51.2	56.1	57.3	56.4
1% stuck, bit lines	71.2	50.8	54.7	56.3	52.2
ADC 7 bits	88.4	87.8	89.0	88.2	88.5
ADC 6 bits	88.0	81.7	89.7	88.8	88.4
ADC 5 bits	71.0	56.5	70.6	70.9	77.3
IR drop $r = 3 \times 10^{-4}$	89.4	89.1	89.7	89.3	89.6
IR drop $r = 10^{-3}$	85.3	89.9	86.6	86.4	86.9
IR drop $r = 3 \times 10^{-3}$	70.1	77.9	73.4	72.6	74.7